

鎖與交易

朱克剛



交易

- 左邊的人口袋少了一筆錢
- 右邊的人口袋會多一筆錢
- 兩個人合起來鈔票總數不變
- 這筆交易中有兩個資料更新指令
- 結果必須同時更新成功否則同時失敗，不可以部分成功部分失敗



原子性

- 原子性（ Atomicity ）：在交易中的各個異動指令合起來算一個，要就全部成功不然全部失敗
- 如果失敗，已經異動的資料必須恢復到交易之前的狀態（ 吃了後悔藥，資料恢復原狀 ）

一致性

- 一致性（ Consistency ）：資料在交易前後必須滿足設定的條件，如果不滿足，交易結果就必須設定失敗
- 例1：A轉錢給B，完成後A減少得要剛好等於B增加的
- 例2：演唱會門票，賣出去的票數必須等於擁有的票數，不能超賣

持久性

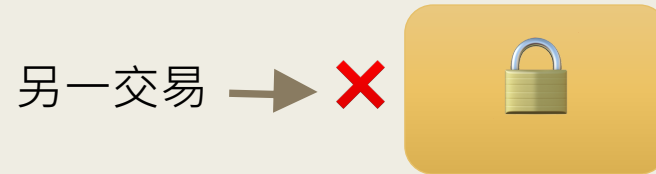
- 持久性（Durability）：一旦交易完成，就沒有後悔藥可以吃了

```
begin transaction;  
-- SQL 指令寫這  
  
rollback; -- 交易失敗  
commit; -- 交易成功
```

- 未啟動交易的異動指令，執行完就自動 commit，稱為 auto-commit。

隔離性

- 隔離性（ Isolation ）：各個交易在自己的地盤中執行，不會互相干擾
- 隔離性必須靠鎖才能達成



- 常聽到資料要滿足 **ACID**，意思就是**A**原子性、**C**一致性、**I**隔離性與**D**耐久性，使用交易才能達到這個要求

鎖的種類

鎖種類	說明
S	共享鎖
X	獨佔鎖、排他鎖
U	修改鎖。修改資料前上U鎖，實際寫入時改X鎖
IS	意圖共享鎖
IX	意圖獨佔鎖
IU	意圖修改鎖
...	https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-lock-transact-sql?view=sql-server-ver16

觀察工具

```
CREATE VIEW lockinfo AS

SELECT
    object_name(b.object_id) as TableName,
    resource_type,
    request_mode,
    request_session_id
FROM sys.dm_tran_locks as a join sys.partitions as b
    on a.resource_associated_entity_id = b.hobt_id
```


測試

- 開兩個頁籤
- 第一個頁籤執行下列指令

```
begin transaction  
update UserInfo set cname = '珠小妹' where uid = 'A04';
```

- 第二個頁籤執行下列指令

```
select * from UserInfo
```

查詢要上 S 鎖

- 結果：第二個頁籤的指令被阻擋，因為第一個頁籤的交易未結束
 - 此時下指令可以觀察到資料出現 X 鎖

處理堵塞

- 找到哪一個 request session id 造成堵塞
- 下指令刪掉他，刪掉的指令會產生交易失敗

`kill SPID`

鎖定時間

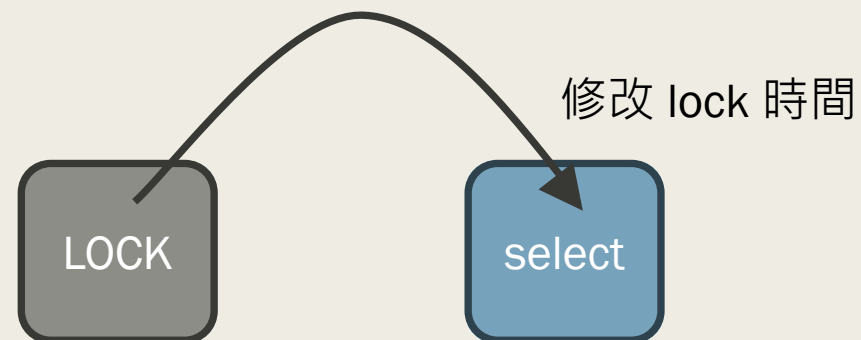
- 查詢鎖定時間

```
print @@lock_timeout
```

- 設定鎖定逾時時間，單位毫秒

```
set lock_timeout 5000
```

- 目的：避免出現死結狀態



範例

- 第一個頁籤執行下列指令

```
begin transaction
  update UserInfo set cname = '珠小妹' where uid = 'A04';
```

- 第二個頁籤執行下列指令

```
set lock_timeout 5000
begin try
  select * from UserInfo
end try
begin catch

end catch
```

讀到出現錯誤為止
例如：A04以後都不會出現

超賣問題 – 通用解法

- 當多人要同時修改同一筆資料，並且該資料會透過判斷式來決定是否修改時，必須使用交易，例如商品有數量限制時。
- 總共只有10樣東西可賣，但最後發現賣超過數量了
- 原因在於「取出數量」、「檢查數量」、「決定賣出」為三個獨立程序，在多工環境下產生的必然現象
- 解決方式：將這三個程序合併成一個程序，然後一次只有一個人執行



注意效能問題

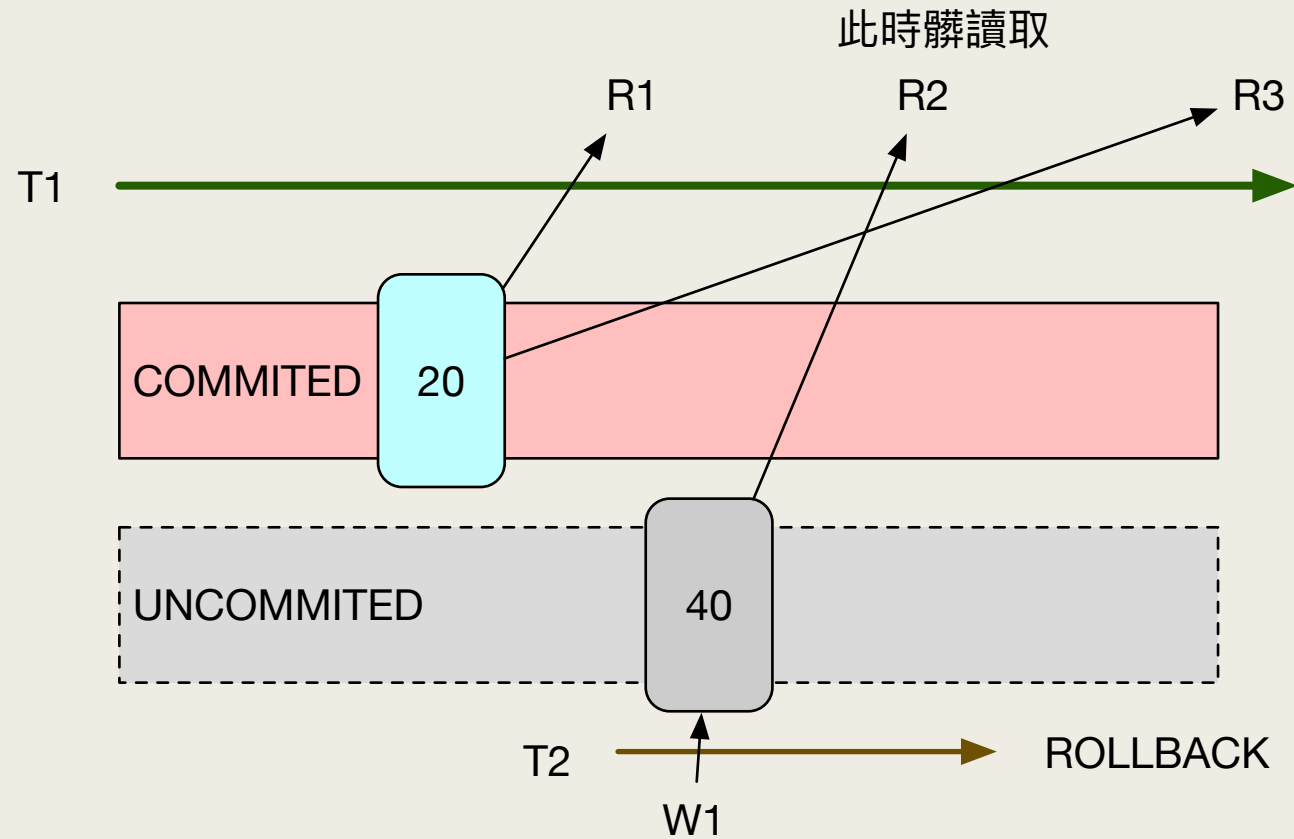
- 臨界區間一次只能服務一個人，資料庫效能再強都沒用
- 臨界區間從誕生到消失的持續時間越短越好
- 不要過度膨脹臨界區間範圍，例如為了幾筆資料卻把整個資料表鎖住
- 查詢範圍必須避開臨界區間，否則原本很快的查詢也會變的很慢

隔離等級

髒讀取：讀到另外一個交易已更動但尚未 commit 的資料

- **READ UNCOMMITTED**：不管資料有沒有上鎖都可以讀取，但可能髒讀取
`set transaction isolation level READ UNCOMMITTED`
- **READ COMMITTED** (SQL Server 預設)
 - 確保讀到的資料都是已確認的
- **REPEATABLE READ**
 - 交易過程中會對讀取資料上 S 鎖，直到交易結束才解鎖，確保交易過程中多次查詢的資料結果都是一致的
- **SERIALIZABLE**
 - 確保一群交易依序執行，不會多個交易同時執行導致讀寫交錯
 - 因為需要更多的鎖來控制執行順序，因此會影響效率
- **SNAPSHOT**：必須開啟後才能使用
`ALTER DATABASE AddressBook SET ALLOW_SNAPSHOT_ISOLATION ON`
`ALTER DATABASE AddressBook SET READ_COMMITTED_SNAPSHOT ON`
- 文件：<https://learn.microsoft.com/zh-tw/sql/t-sql/statements/set-transaction-isolation-level-transact-sql?view=sql-server-ver16>

READ UNCOMMITTED



READ UNCOMMITTED – 測試

- 在第一個頁籤 (T1) 執行

```
set transaction isolation level READ UNCOMMITTED  
begin transaction
```

```
select 'r1', * from UserInfo where uid = 'A01'
```

王大明

```
waitfor delay '0:0:10'
```

```
select 'r2', * from UserInfo where uid = 'A01'
```

David

```
waitfor delay '0:0:10'
```

```
select 'r3', * from UserInfo where uid = 'A01'
```

王大明

```
commit;
```

- 在 r1 與 r2 之間，第二個頁籤 (T2) 執行

```
begin transaction
```

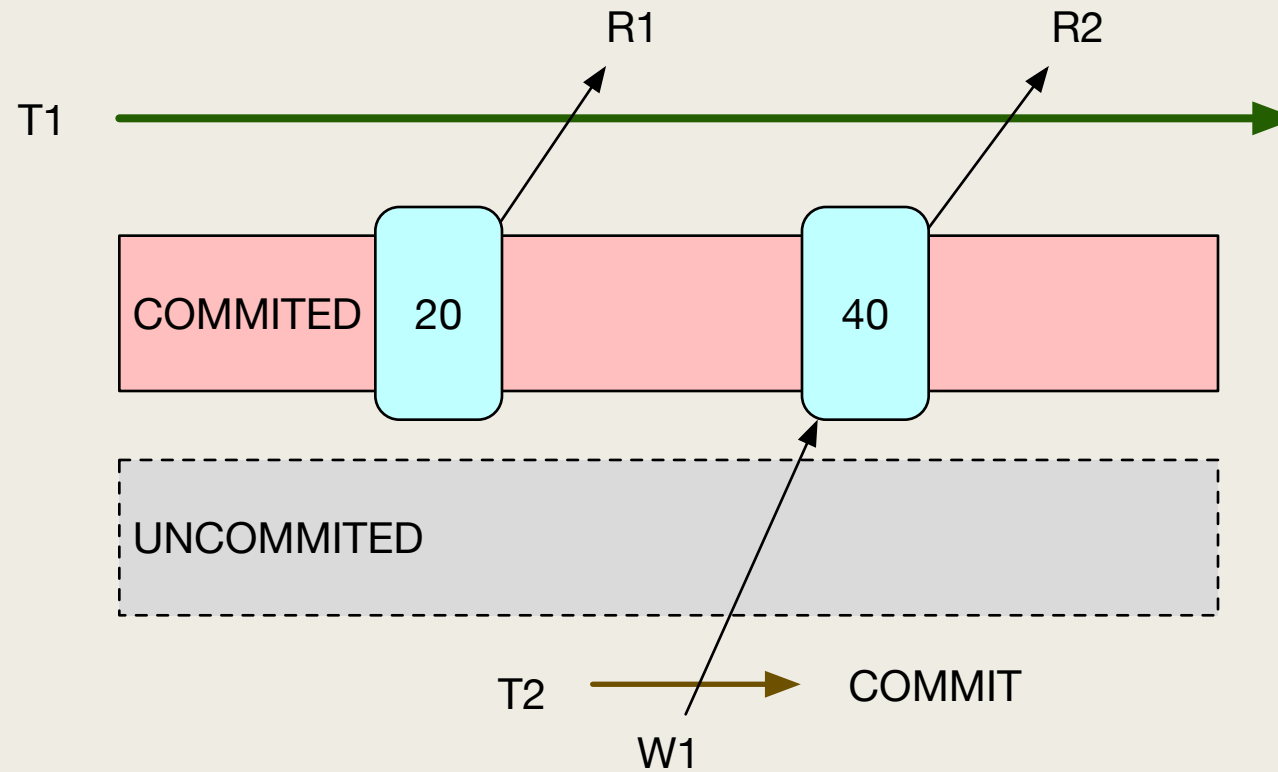
```
update UserInfo set cname = 'David' where uid = 'A01';
```

```
waitfor delay '0:0:10'
```

```
rollback
```

在這個時間點執行

READ COMMITTED



READ COMMITTED – 測試

- 在第一個頁籤 (T1) 執行

```
set transaction isolation level READ COMMITTED
```

```
begin transaction
```

```
select 'r1', * from UserInfo where uid = 'A01'
```

王大明

```
waitfor delay '0:0:10'
```

```
select 'r2', * from UserInfo where uid = 'A01'
```

David

```
commit;
```

- 在 r1 與 r2 之間，第二個頁籤 (T2) 執行

```
begin transaction
```

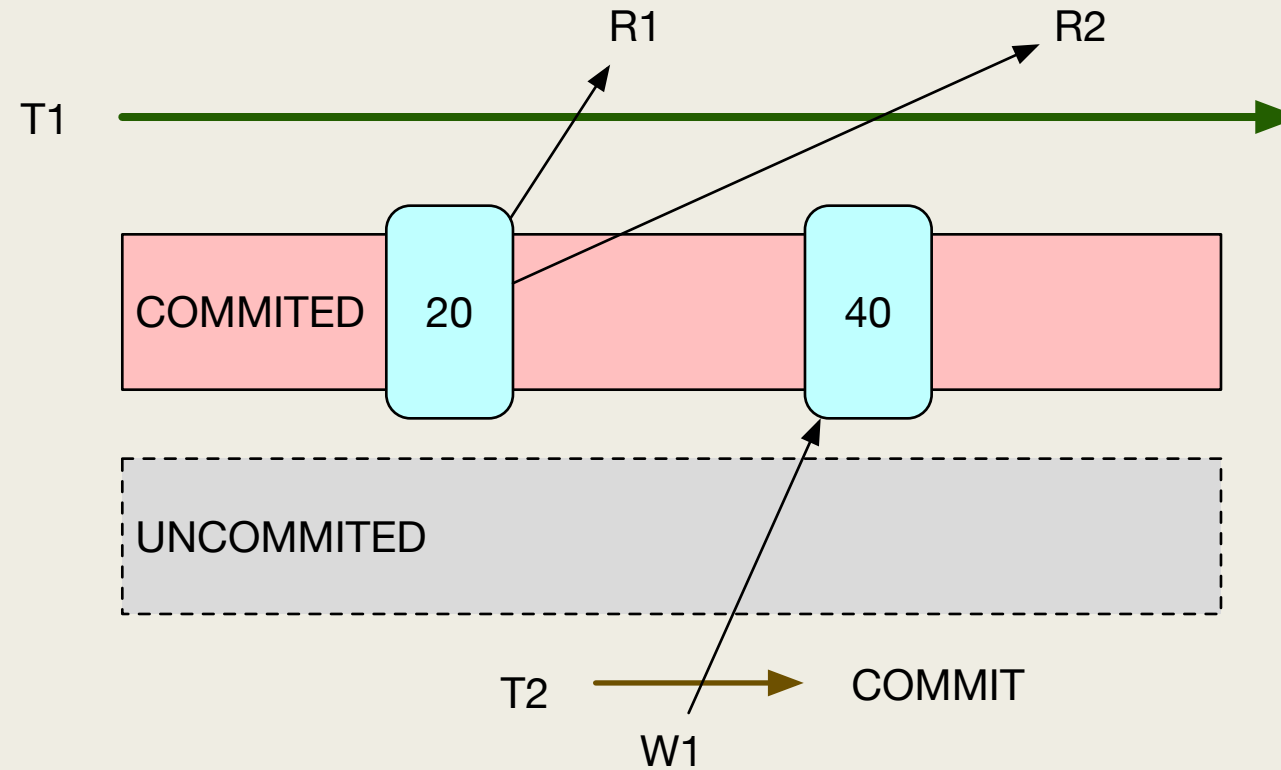
```
update UserInfo set cname = 'David' where uid = 'A01';
```

```
waitfor delay '0:0:10'
```

```
commit
```

在這個時間點執行

REPEATABLE READ



REPEATABLE READ – 測試

- 在第一個頁籤 (T1) 執行

```
set transaction isolation level REPEATABLE READ
begin transaction
  select 'r1', * from UserInfo where uid = 'A01'
  waitfor delay '0:0:10'
  select 'r2', * from UserInfo where uid = 'A01'
commit;
```

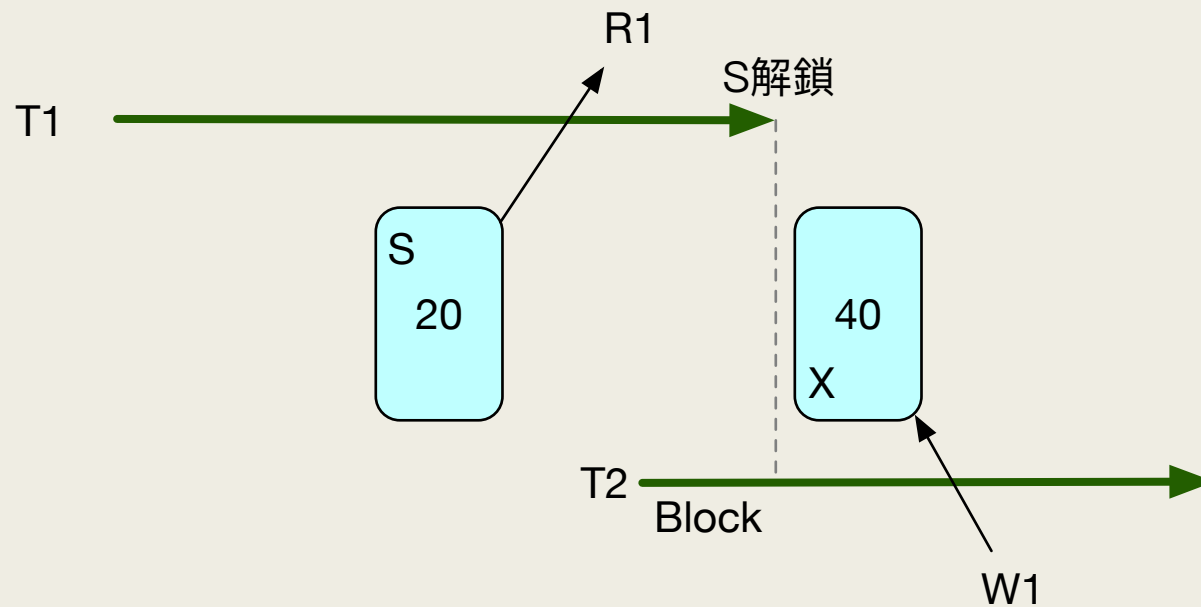
- 在 r1 與 r2 之間，第二個頁籤 (T2) 執行

```
update UserInfo set cname = 'Tom' where uid = 'A01'
```

- 結果：T1 的兩次查詢結果都一樣

SERIALIZABLE

- 透過各種鎖，確保各交易間讀寫不交錯
- S 鎖會等交易結束後才解鎖



SERIALIZABLE – 測試

■ T1

```
set transaction isolation level SERIALIZABLE;  
begin tran  
select * from UserInfo where uid = 'A01';  
waitfor delay '0:0:10';  
commit;
```

此處執行T2



■ T2

```
set transaction isolation level SERIALIZABLE;  
begin tran  
update UserInfo set cname = 'Tom' where uid = 'A01';  
commit;
```

到 commit 才會解S鎖

等 T1 結束才能 update

SNAPSHOT

- 交易在快照中進行，各交易彼此之間均不干擾，也不會因為其他交易未解鎖而導致阻塞

- T1 執行

不會因為 T2 未完成，
導致 T1 進入等待

```
set transaction isolation level SNAPSHOT  
begin transaction
```

此處執行T2

```
select 'r1', * from UserInfo where uid = 'A01'  
waitfor delay '0:0:10'  
select 'r2', * from UserInfo where uid = 'A01'  
commit;
```

- T2 在 r1 與 r2 間執行

```
set transaction isolation level SNAPSHOT  
begin tran
```

```
update UserInfo set cname = 'David' where uid = 'A01';
```

此交易未完成

SNAPSHOT 限制

- SNAPSHOT 會先複製一份資料到 tempdb，然後在複製的資料中讀取
- 雖然可以更新資料，但不可以在已讀取後更新別的交易已經更新的資料
- T1

```
set transaction isolation level snapshot;
```

```
begin tran
```

```
select * from UserInfo where uid = 'A01';
```

這裡讀取了資料

此處執行T2

```
waitfor delay '0:0:10';
```

```
update UserInfo set cname = 'Tom' where uid = 'A01';
```

```
commit;
```

更新別人已更新的同一筆資料
會收到錯誤

- T2

```
set transaction isolation level snapshot;
```

```
begin tran
```

```
update UserInfo set cname = 'Mei' where uid = 'A01';
```

```
commit;
```

手動設定鎖 – SQL Server 專有

■ NOLOCK

- 讀取資料時不設定S鎖，若此時資料已經有X鎖，照樣可以讀取，目的是增加讀取效率，但可能髒讀取
- 等同隔離等級中的 READ UNCOMMITTED

■ READPAST

- 跟 NOLOCK 類似但讀取到的資料不包含已上鎖資料，因此不會髒讀取，但資料會少

■ UPDLOCK / XLOCK

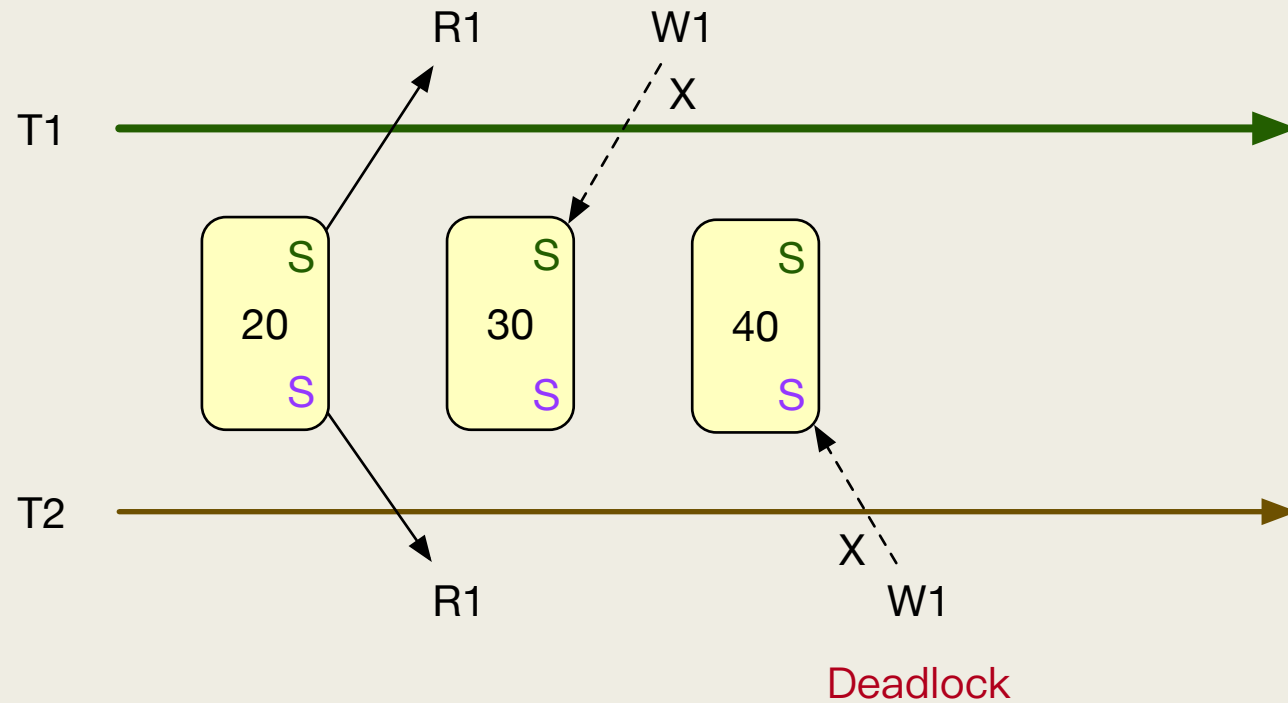
- 讀取資料時可對資料上修改鎖或獨佔鎖，直到交易結束

■ ROWLOCK

- 某些情況為避免觸發更上層鎖造成效率銳減，因此可設定僅對行上鎖

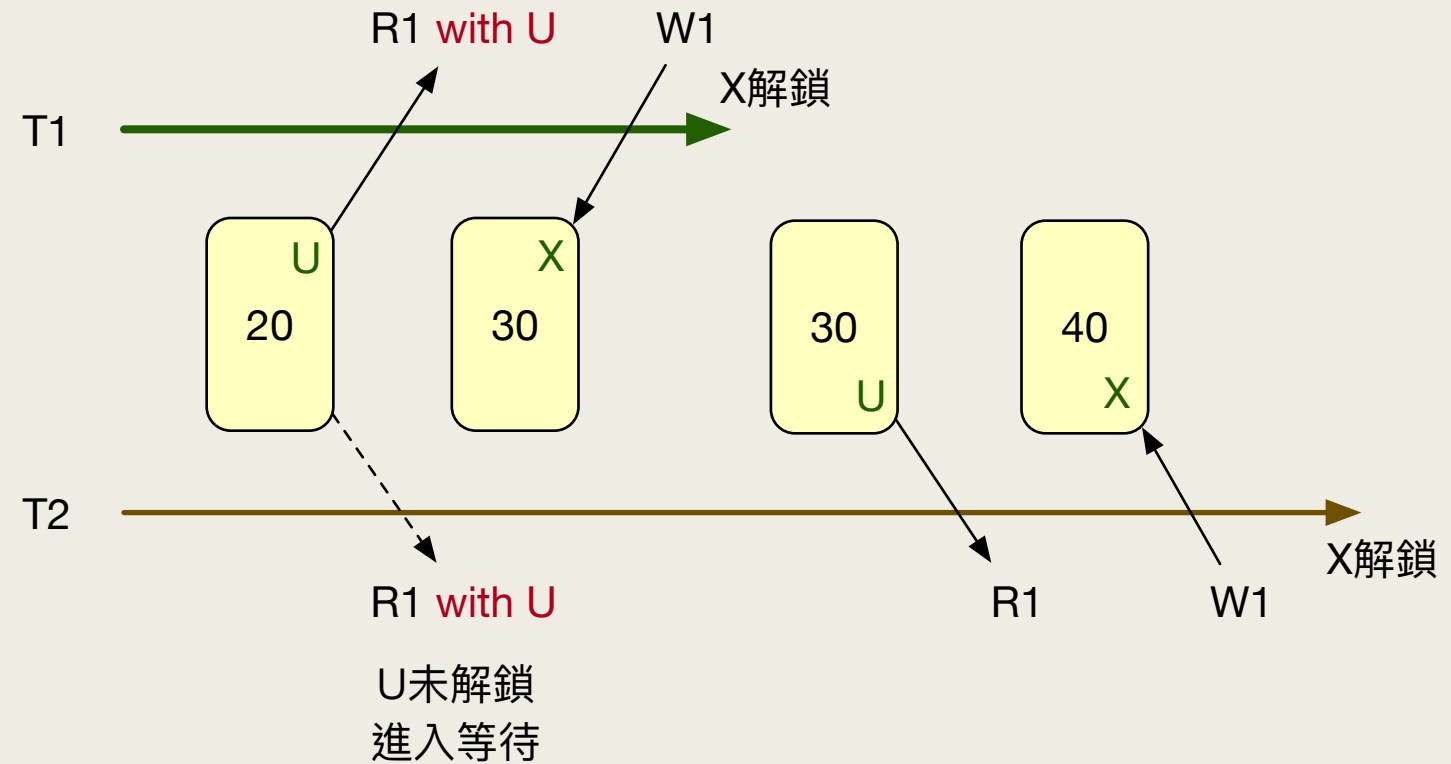
死結 – U 鎖可解

- 在 T1 的 W1 時，需要等 T2 commit 才能解鎖 T2 上的 S 鎖
- 在 T2 的 W1 時，需要等 T1 commit 才能解鎖 T1 上的 S 鎖
- 此時 T1、T2 進入互相等待狀態，形成死結 (SQL Server可防止此種死結)



U鎖

- 有 S 鎖的資料可上一個 U 鎖，此時 S 與 U 可共存
- 只能有一個 U 鎖



U鎖 - 測試

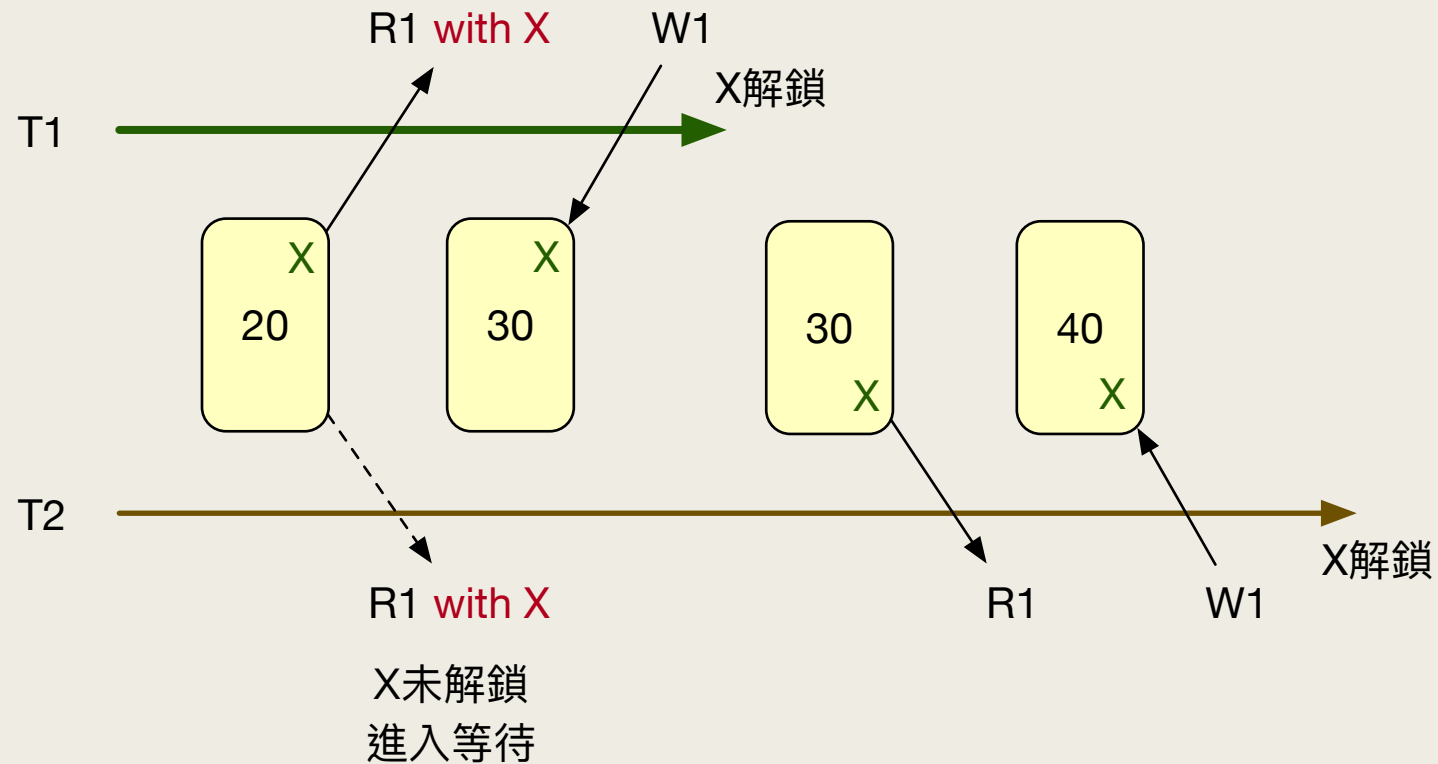
■ T1

```
set transaction isolation level serializable;  
begin tran  
    select * from UserInfo with (updlock) where uid = 'A01';  
    waitfor delay '0:0:10';  
    update UserInfo set cname = 'David' where uid = 'A01';  
    commit;
```

■ T2

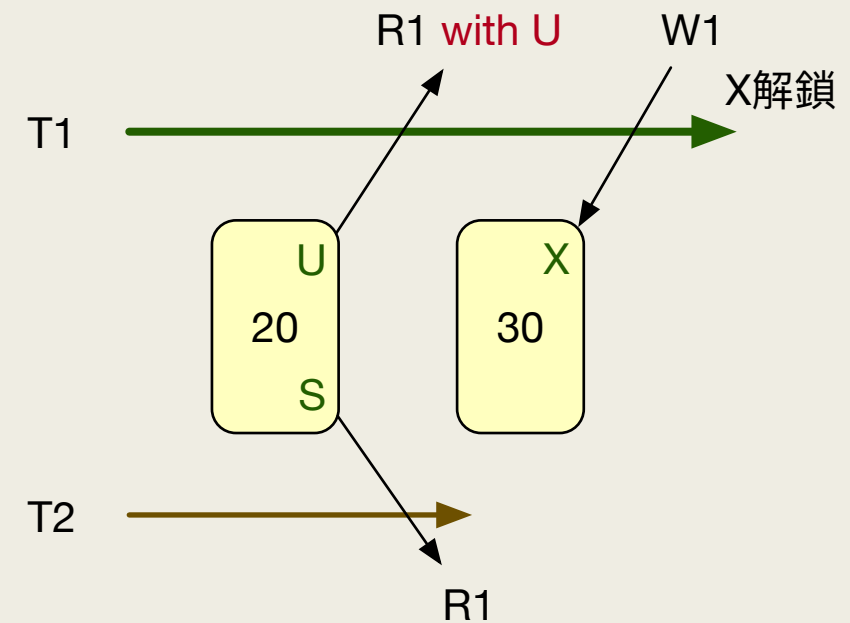
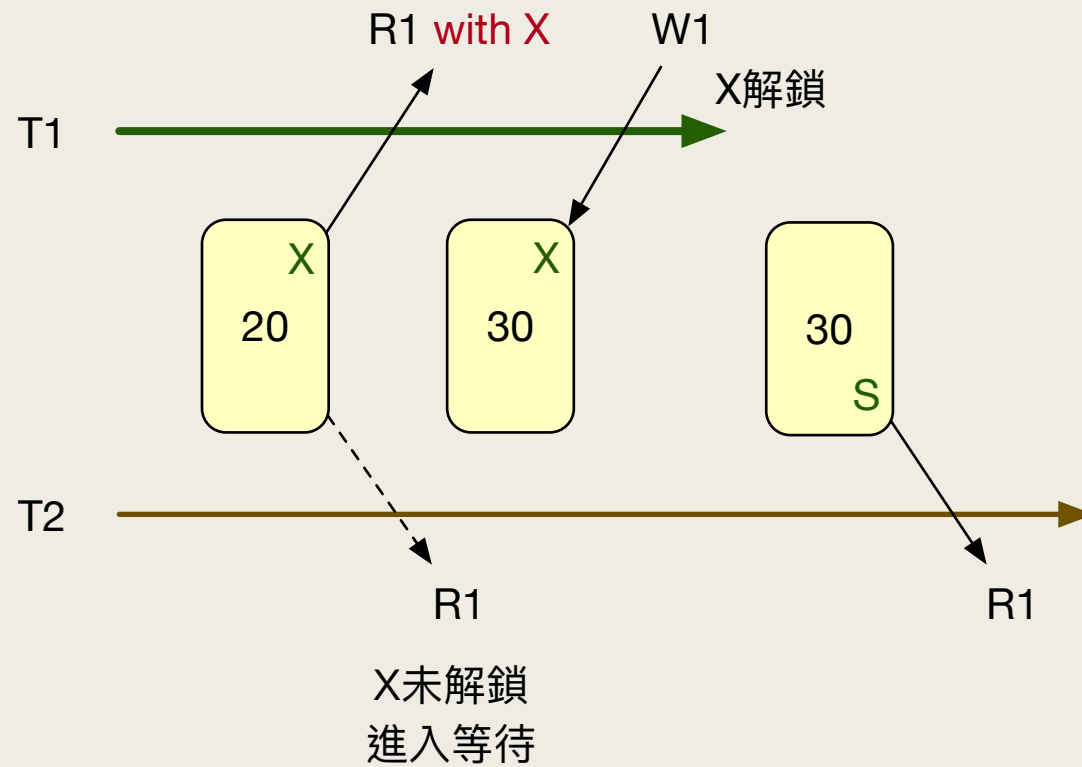
```
set transaction isolation level serializable;  
begin tran  
    select * from UserInfo with (updlock) where uid = 'A01';  
    update UserInfo set cname = 'Tom' where uid = 'A01';  
    commit;
```

可以用X鎖預防死結嗎？



可以，但有時沒有效率

T2只是讀資料時會被T1卡住



死結 1/3

- 互相等對方解鎖時就形成死結
- 首先建立兩個測試用資料表

```
drop table if exists TAA  
drop table if exists TBB
```

```
select 1 as id, 0 as n into TAA  
select 1 as id, 0 as n into TBB
```


死結 2/3

- 在第一個頁籤執行，先 TAA 再 TBB

```
begin TRAN
  update TAA set n = n + 1 where id = 1
  waitfor delay '00:00:05'
  update TBB set n = n + 1 where id = 1
  commit
```

此行造成死結，導致
此交易失敗

- 在第二個頁籤執行，先 TBB 再 TAA

```
begin TRAN
  update TBB set n = n + 1 where id = 1
  waitfor delay '00:00:05'
  update TAA set n = n + 1 where id = 1
  commit
```

死結 3/3

- 手動上 U 鎖
- 在兩個頁籤的交易開始時都對資料行加上 U 鎖

```
begin TRAN
    select * from TAA with (updlock) where id = 1
    select * from TBB with (updlock) where id = 1
```

- 兩個頁籤都執行完畢後，應該 n 的值是 2

	id	n
1	1	2

超賣問題 – SQL Server 專屬作法

```
create proc buy2 as
begin
    declare @quantity int
    begin tran
        select @quantity = quantity from Product with (updlock) where pid = 1
        waitfor delay '0:0:5'
        if @value > 0
            update Product set quantity = quantity - 1 where pid = 1
        commit
    end
end
```